KCF Object Tracking

1. Function Description

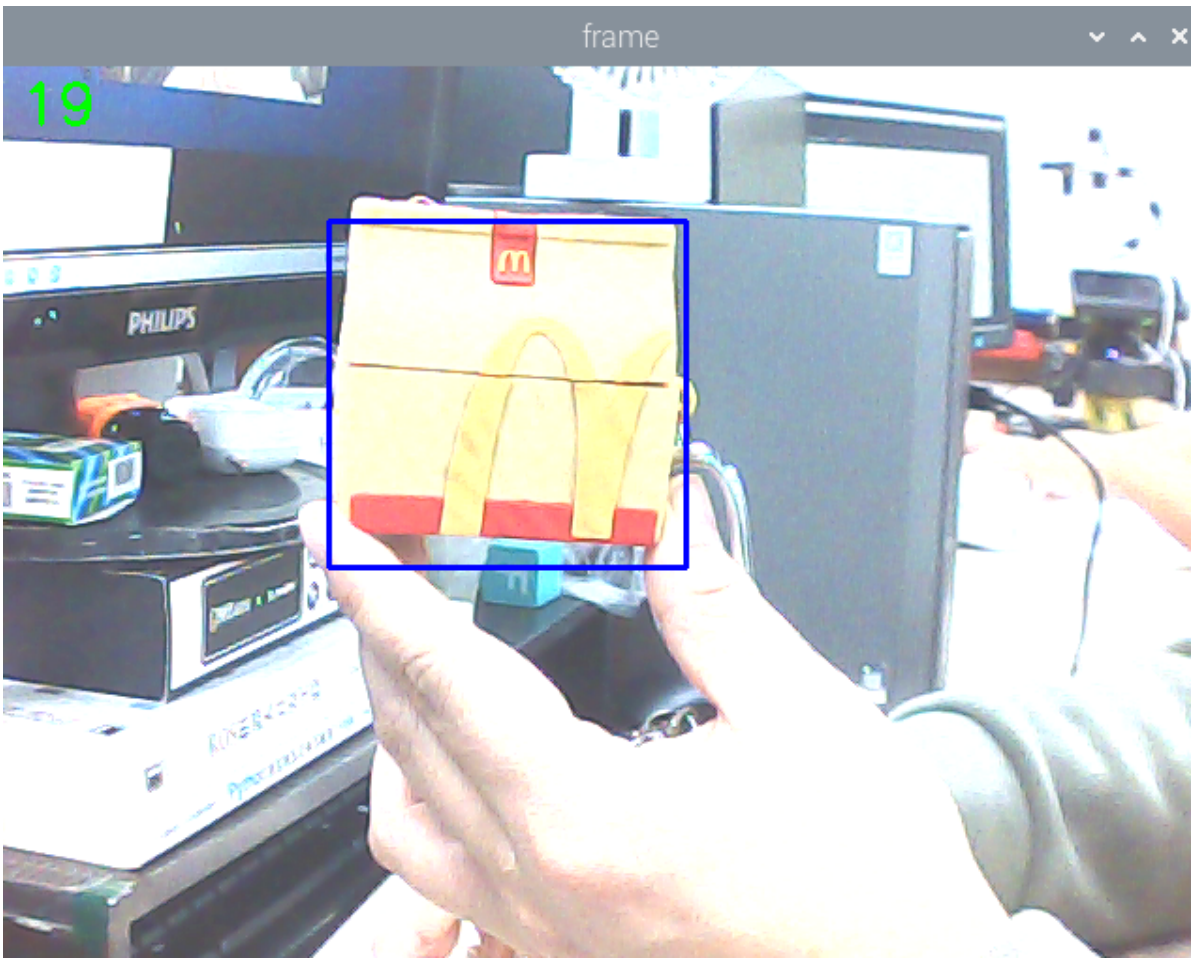
After the program runs, the robotic arm will lift up and enter tracking posture. Hold an object and place it within the camera's field of view. Then use the mouse to draw a box around the object to be tracked. Finally, press the spacebar, and the robotic arm will adjust its posture to keep the object's center near the center of the image. Slowly move the object, and the robotic arm will follow the object's movement.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start object KCF tracking program node
ros2 run dofbot_follow KCF_follow
```

After the program runs successfully, the robotic arm will move to its initial posture. Hold an object and place it within the camera's field of view. Use the mouse to draw a box around the object to be tracked, as shown in the figure below:



Then press the spacebar, and the robotic arm will adjust its posture to keep the object's center within the center range of the image at all times. Slowly move the object, and the robotic arm will follow the object's movement.

3. Code Analysis

3.1. KCF_follow

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_follow/dofbot_follow/KCF_follow.py
```

Import necessary library files:

```
import cv2
import rclpy
from rclpy.node import Node
import numpy as np
from sensor_msgs.msg import Image
from std_msgs.msg import Bool ,Int16MultiArray,String
from cv_bridge import CvBridge
import cv2 as cv
import time
from sensor_msgs.msg import Image
from std_msgs.msg import Int32,UInt16
from Arm_Lib import Arm_Device
import dofbot_follow.PID as PID
from  dofbot_follow.Track_Lib import *
import os
```

Create robotic arm control object and control the robotic arm to move to initial posture:

```
self.Arm = Arm_Device()
self.init_joints = [90, 150, 0, 20, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Initialize PID:

```
self.xservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
```

Create subscriber to image topic:

```
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages and convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'bgr8')
    result_image = np.copy(rgb_image)
    # Wait for key input
    action = cv.waitKey(10) & 0xFF
    # If spacebar is pressed, self.start_follow value becomes True, indicating
tracking can start
    if action == 32:
        self.start_follow = True
```

```
    result_image = cv.resize(result_image, (640, 480))
    # Pass image as parameter to process function
    result_frame = self.process(result_image,action)
    # If object center point coordinates are not 0 and self.start_follow value is
True, then tracking can proceed
    if self.cx!=0 and self.cy!=0 and self.start_follow == True:
        # If the object's center coordinates x or y are not within the center
range, execute XY_track function with current object center coordinates as
parameters
        if (abs(self.cx-320) >30 or abs(self.cy-240)>30) :
            self.XY_track(self.cx,self.cy)
            print("Tracking")
            print("-------------------------------------")
        cv2.circle(result_frame, (int(self.cx),int(self.cy)), 10, (255,255,0),
thickness=None, lineType=None, shift=None)
    cur_time = time.time()
    fps = str(int(1/(cur_time - self.pr_time)))
    self.pr_time = cur_time
    cv2.putText(result_frame, fps, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)
    cv.imshow(self.windows_name, result_frame)
```

process function:

```
def process(self, rgb_img, action):
    rgb_img = cv.resize(rgb_img, (640, 480))
    if action == ord('r') or action == 114: self.Reset()
    # If current tracking state is 'init', it means the object to be tracked has
not been selected with mouse yet, so execute "detect mouse and draw object box"
program
    if self.Track_state == 'init':
        cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
        cv.setMouseCallback(self.windows_name, self.onMouse, 0)
        if self.select_flags == True:
            cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
            cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)
    # If state is not 'init', it means the object to be tracked has been
selected
    if self.Track_state != 'init':
        if self.gTracker_state == True:
            # Get mouse selection box coordinates
            Roi = (self.Roi_init[0], self.Roi_init[1], self.Roi_init[2] -
self.Roi_init[0], self.Roi_init[3] - self.Roi_init[1])
            # Create a tracker with tracking type self.tracker_type, which is
KCF
            self.gTracker = Tracker(tracker_type=self.tracker_type)
            # Initialization work
            self.gTracker.initWorking(rgb_img, Roi)
            self.gTracker_state = False
        # Start executing tracking program and get tracking box coordinates
        rgb_img, (targBegin_x, targBegin_y), (targEnd_x, targEnd_y) =
self.gTracker.track(rgb_img)
        # Calculate object center coordinates from tracking box coordinates and
assign to self.cx and self.cy
        center_x = targEnd_x / 2 + targBegin_x / 2
        center_y = targEnd_y / 2 + targBegin_y / 2
        self.cx = center_x
```

```
        self.cy = center_y
        print(self.cx)
        print(self.cy)
    return rgb_img
```